

DeliverExpress — Documentación del Proyecto

Sep 26, 2026 · @Kirito

1. Información general

DeliverExpress es el proyecto grupal de la materia Bases de Datos: un sistema de gestión de delivery cuyo núcleo es una base de datos PostgreSQL que corre localmente en cada computadora.

Campo	Detalle
Materia	Bases de Datos
Proyecto	DeliverExpress — Gestión de Delivery
Motor de base de datos	PostgreSQL, instalado localmente (no se permiten servicios en la nube como Supabase)
Herramienta de administración	pgAdmin 4
Integrantes	5 (nombres en la sección 14)
Fecha de entrega	Por definir
Profesor	Por completar

2. Descripción del problema

DeliverExpress conecta restaurantes, repartidores y clientes, pero hoy gestiona todo con hojas de cálculo y WhatsApp. El mes pasado tuvo que devolver el 10% de los pedidos por errores de asignación.

Situación actual según la entrevista:

Elemento	Cantidad actual
Restaurantes asociados	20
Repartidores	35
Coordinadores que atienden llamadas	3

Problemas reportados por el cliente:

- Se pierden pedidos y se mezclan direcciones.
- Los repartidores reciben órdenes con datos incorrectos.
- No se mide cuánto tarda cada etapa (preparación, retiro, entrega).
- La asignación de repartidores es manual y poco eficiente.
- No hay registro ordenado de pagos, comisiones, envíos y propinas.
- No existe una vista central de pedidos activos para los coordinadores.

3. Objetivos

Objetivo general: diseñar e implementar una base de datos relacional en PostgreSQL, con aplicación web asociada, que gestione pedidos, restaurantes, repartidores, clientes, pagos y calificaciones de DeliverExpress.

Objetivos específicos:

1. Modelar los datos del negocio en un diagrama entidad-relación normalizado hasta tercera forma normal (3FN).
2. Implementar el esquema con restricciones de integridad (PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE, NOT NULL).
3. Registrar cada cambio de estado del pedido con fecha y hora para medir el tiempo de cada etapa.
4. Programar en la base de datos la asignación automática del repartidor más cercano, disponible y mejor calificado.
5. Calcular distancias y costos de envío con funciones dentro de PostgreSQL.
6. Registrar pagos, comisión de la plataforma (15%), pago al restaurante, envío y propinas.
7. Gestionar calificaciones entre clientes, repartidores y restaurantes, con promedios actualizados automáticamente.
8. Ofrecer vistas y reportes para el panel de coordinadores y para la toma de decisiones.
9. Desarrollar un backend y un frontend que usen la base de datos de forma segura.

4. Alcance

El proyecto entrega un sistema funcional de demostración: la base de datos es completa y el frontend cubre los flujos principales de cada actor. Pagos, GPS y mapas se simulan.

Incluye	No incluye (se simula o queda fuera)
Base de datos completa en PostgreSQL local con esquema, restricciones, funciones, triggers, vistas, roles y respaldos	Base de datos en la nube o servicios como Supabase, Firebase o similares
Registro de restaurantes, menús, zonas de cobertura y horarios	App móvil nativa: se hace una aplicación web adaptable al celular
Flujo completo del pedido con historial de estados y tiempos por etapa	Cobro real con tarjeta: el pago se simula y no se guardan números de tarjeta
Asignación automática de repartidores, aceptación y rechazo	GPS real del repartidor: la ubicación se simula con un script que mueve coordenadas
Cálculo de distancia (línea recta) y tarifa de envío	Distancia por calles reales: se usa distancia en línea recta (fórmula de Haversine)
Pagos, comisiones, envíos y propinas registrados	Facturación fiscal
Calificaciones cruzadas y promedios	Recomendaciones con inteligencia artificial: se usan consultas SQL sobre el historial
Panel de coordinadores con mapa y pedidos activos	Notificaciones push o SMS
Recomendaciones simples basadas en el historial del cliente	

5. Actores y roles del sistema

El sistema tiene cinco tipos de usuario; cada uno ve una interfaz distinta y tiene permisos distintos.

Actor	Qué hace en el sistema	Interfaz
Cliente	Elige restaurante, arma el pedido, paga, agrega propina, sigue el pedido en el mapa y califica	App de cliente
Restaurante	Gestiona su menú y horario, acepta y prepara pedidos, marca "listo para retirar", ve quién retira y cuándo	Panel de restaurante

Actor	Qué hace en el sistema	Interfaz
Repartidor	Cambia su disponibilidad, acepta o rechaza pedidos, marca retiro y entrega, califica al cliente	Panel de repartidor
Coordinador	Supervisa pedidos activos, repartidores disponibles y tiempos; puede reasignar pedidos manualmente	Panel de control (dashboard)
Administrador	Gestiona zonas, categorías, tarifas, parámetros y usuarios	Panel de administración

6. Requerimientos funcionales

Cada requerimiento sale de la entrevista con el cliente y se asocia a un área del sistema.

ID	Requerimiento	Área
RF-01	Registrar restaurantes con nombre, dirección, teléfono, coordenadas, categoría y tiempo estimado de preparación	Restaurantes
RF-02	Registrar el horario de atención de cada restaurante por día de la semana	Restaurantes
RF-03	Definir las zonas de cobertura de cada restaurante (un restaurante cubre varias zonas)	Restaurantes
RF-04	Gestionar el menú de productos con precio y disponibilidad	Restaurantes
RF-05	Registrar clientes con nombre, teléfono y una o varias direcciones con coordenadas	Clientes
RF-06	Registrar repartidores con nombre, teléfono, vehículo (bicicleta, moto, auto), zona, disponibilidad y calificación promedio	Repartidores
RF-07	Crear un pedido con varios productos, validando que el restaurante esté abierto y cubra la zona del cliente	Pedidos
RF-08	Manejar los estados del pedido: recibido, en preparación, listo para retirar, en camino, entregado y cancelado	Pedidos

ID	Requerimiento	Área
RF-09	Guardar fecha y hora de cada cambio de estado y calcular el tiempo de cada etapa	Pedidos
RF-10	Calcular la distancia entre restaurante y cliente y el costo del envío según esa distancia	Distancias y tarifas
RF-11	Asignar automáticamente el repartidor disponible más cercano al restaurante, priorizando buena calificación	Asignación
RF-12	Permitir al repartidor aceptar o rechazar un pedido; si rechaza, ofrecerlo al siguiente candidato	Asignación
RF-13	Bajar la prioridad de los repartidores con muchos rechazos	Asignación
RF-14	Registrar el pago con tarjeta al crear el pedido (simulado)	Pagos
RF-15	Calcular y registrar la comisión de la plataforma (15%), el monto del restaurante, el envío y la propina del repartidor	Pagos
RF-16	Permitir al cliente agregar propina	Pagos
RF-17	Permitir calificar: cliente al repartidor, cliente al restaurante, repartidor al cliente	Calificaciones
RF-18	Actualizar automáticamente la calificación promedio de repartidores, restaurantes y clientes	Calificaciones
RF-19	Marcar para revisión a repartidores y clientes con calificación baja	Calificaciones
RF-20	Mostrar al cliente la ubicación del repartidor en un mapa durante el envío	Seguimiento
RF-21	Mostrar al restaurante qué repartidor retira el pedido y su hora estimada de llegada	Seguimiento
RF-22	Panel de coordinadores con pedidos activos, repartidores disponibles y tiempos estimados, actualizado en tiempo real	Seguimiento
RF-23	Permitir al coordinador reasignar o cancelar un pedido manualmente	Seguimiento

ID	Requerimiento	Área
RF-24	Mostrar el historial de pedidos del cliente y recomendaciones según lo que más pide	Clientes
RF-25	Generar reportes: tiempos promedio por restaurante y repartidor, ingresos y comisiones, pedidos cancelados	Reportes
RF-26	Iniciar sesión con usuario y contraseña según el rol	Seguridad

7. Requerimientos no funcionales y restricciones

La restricción principal es que la base de datos sea PostgreSQL y corra localmente en la computadora; el resto de las herramientas es flexible siempre que se integren.

ID	Requerimiento	Tipo
RNF-01	La base de datos es PostgreSQL, instalada localmente en cada PC. No se usan servicios en la nube (Supabase, Neon, Firebase, etc.)	Restricción obligatoria
RNF-02	Todos los integrantes usan PostgreSQL 16	Restricción
RNF-03	La base se administra con pgAdmin 4	Restricción
RNF-04	La base debe poder reconstruirse desde cero ejecutando los scripts SQL del repositorio en orden	Mantenibilidad
RNF-05	La lógica crítica del negocio (estados, asignación, cálculos, promedios) vive en la base de datos, no solo en el backend	Integridad
RNF-06	Los montos de dinero usan NUMERIC(10,2), nunca tipos de punto flotante	Integridad
RNF-07	Las fechas usan TIMESTAMPTZ para evitar errores de zona horaria	Integridad
RNF-08	Las contraseñas se guardan cifradas (hash con bcrypt o la extensión pgcrypto)	Seguridad
RNF-09	No se guardan números completos de tarjeta; solo los últimos 4 dígitos y una referencia simulada	Seguridad

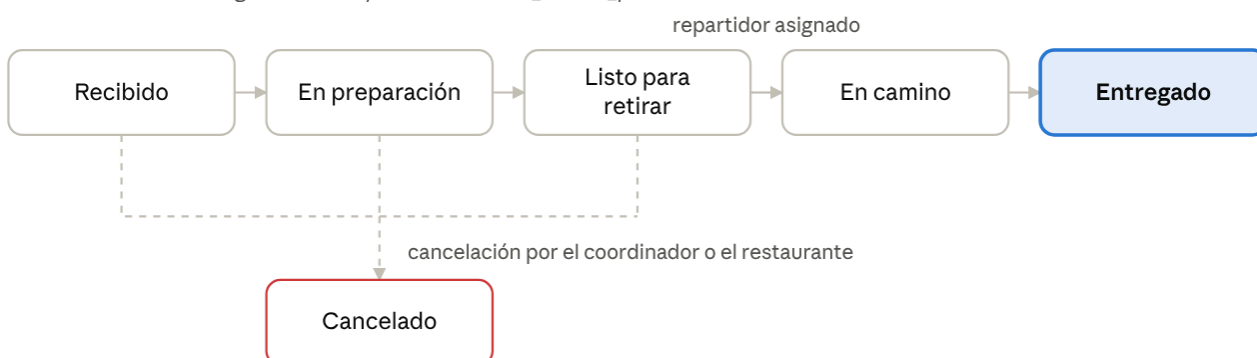
ID	Requerimiento	Tipo
RNF-10	El backend se conecta con un usuario de base de datos con permisos limitados, no con el superusuario postgres	Seguridad
RNF-11	Las consultas desde el backend son parametrizadas para evitar inyección SQL	Seguridad
RNF-12	Dos coordinadores no pueden asignar el mismo repartidor a dos pedidos al mismo tiempo (control de concurrencia)	Concurrencia
RNF-13	El panel de coordinadores se actualiza en menos de 5 segundos tras un cambio de estado	Rendimiento
RNF-14	La interfaz funciona en navegador de escritorio y de celular	Usabilidad

8. Reglas de negocio

Estas reglas se implementan dentro de PostgreSQL (CHECK, triggers y funciones), para que ningún programa pueda saltarlas. Las marcadas con (S) eran supuestos que el grupo ya confirmó (sección 9).

Estados del pedido

Cada cambio de estado guarda fecha y hora en historial_estado_pedido



estados del pedido · 5 estados + cancelado

Un pedido solo avanza hacia adelante; no puede saltar estados ni volver atrás. Solo se cancela antes de estar "en camino".

ID	Regla
RN-01	Transiciones válidas: recibido → en preparación → listo para retirar → en camino → entregado. Cancelado solo desde los tres primeros estados
RN-02	Cada cambio de estado se registra automáticamente con fecha y hora (trigger)
RN-03	Un pedido solo se crea si el restaurante está activo, abierto en ese horario y cubre la zona de la dirección de entrega
RN-04	Todos los productos de un pedido pertenecen al mismo restaurante
RN-05	El precio del producto se copia al detalle del pedido; si el restaurante cambia el precio después, los pedidos pasados no cambian
RN-06	El costo de envío se toma de la tabla de tarifas según el rango de distancia en kilómetros
RN-07	Candidatos a asignación: repartidores activos, libres, que trabajan en la zona del restaurante y que no hayan rechazado ya ese pedido
RN-08	Orden de asignación: primero prioridad (normal antes que baja), luego menor distancia al restaurante, luego mayor calificación
RN-09	Un repartidor tiene como máximo un pedido activo; al aceptar pasa a "ocupado" y al entregar vuelve a "libre"
RN-10	(S) Si un repartidor rechaza más del 30% de sus últimas 20 ofertas, su prioridad baja
RN-11	(S) Comisión de la plataforma = 15% del subtotal de productos; el restaurante recibe el 85% restante
RN-12	(S) El repartidor recibe el 100% del costo de envío y el 100% de la propina
RN-13	Total del pedido = subtotal de productos + costo de envío + propina
RN-14	Calificaciones de 1 a 5, solo sobre pedidos entregados, una por tipo por pedido

ID	Regla
RN-15	(S) Repartidor o cliente con promedio menor a 3.0 y al menos 10 calificaciones queda marcado "en revisión"
RN-16	La comisión, los umbrales y los límites se guardan en una tabla de parámetros, no escritos en el código

9. Decisiones tomadas

La entrevista dejaba huecos y dos contradicciones; el grupo ya tomó estas decisiones. Si el profesor indica otra cosa, se cambia aquí y en la tabla de parámetros.

#	Punto de la entrevista	Decisión
1	La entrevista menciona 5 estados; el resumen menciona 4	5 estados de la entrevista más "cancelado"
2	Contradicción: siempre se paga con tarjeta, pero a los mal calificados se les pediría "pagar por adelantado"	El pago es siempre anticipado con tarjeta; el cliente mal calificado queda marcado "en revisión" y el coordinador decide
3	Base del 15% de comisión	15% sobre el subtotal de productos; envío y propina no pagan comisión
4	"El repartidor cobra por envío"	El repartidor recibe el envío completo más la propina
5	"Si rechazan muchos, baja la prioridad"	Más del 30% de rechazo en sus últimas 20 ofertas; configurable
6	"Mala calificación"	Promedio menor a 3.0 con al menos 10 calificaciones; configurable
7	La tarifa varía según la distancia	Tabla de rangos por kilómetro con precios de prueba
8	Cálculo de distancia	Función propia con la fórmula de Haversine (línea recta), sin PostGIS
9	Seguimiento en tiempo real	Triggers con <code>pg_notify</code> + WebSockets de FastAPI; la posición del repartidor se simula con un script

#	Punto de la entrevista	Decisión
10	Pago con tarjeta	Simulado: formulario de prueba, solo se guardan los últimos 4 dígitos
11	Ciudad de los datos de prueba	Puerto Ordaz, estado Bolívar (Venezuela): zonas y coordenadas reales de la ciudad
12	"Qué le gusta" al cliente	Categorías y productos que más pidió, con una vista SQL
13	Tiempo estimado de llegada	Tiempo de preparación + distancia ÷ velocidad promedio del vehículo

Facturación (agregada a propuesta del grupo)

Se agrega facturación al estilo venezolano, tomada del modelo de una integrante y corregida. Es un extra: la entrevista no la pide.

Punto	Decisión
Moneda	Precios en USD. El cliente paga en USD o en bolívares; se guarda la tasa BCV del día y el total en Bs
IVA	16% sobre productos no exentos, envío y comisión; 0% para productos marcados como exentos. La propina no paga IVA
IGTF	3% sobre el total (con IVA y propina) solo si paga en USD
Tasa BCV	La carga el administrador, una por día; si falta la del día se usa la última anterior
Documentos	Factura al cliente automática al entregar; factura mensual de comisión a cada restaurante; nota de crédito para anular; liquidación a repartidores (no fiscal); libro de ventas
Numeración	Número PPPP-NNNNNNNN y control PP-NNNNNN, correlativos y sin huecos
Fuera del alcance	Retenciones de IVA e ISLR, libro de compras, autorización del SENIAT, PDF legal, facturas tipo A/C y CAE (son de Argentina)

10. Arquitectura y herramientas

Arquitectura de tres capas: una aplicación web (frontend) llama a una API (backend), y la API ejecuta funciones y consultas en PostgreSQL local. El editor lo elige cada quien, pero el lenguaje del backend y el framework del frontend deben ser uno solo para todo el grupo.

Flujo de tiempo real: cuando cambia el estado de un pedido o la ubicación de un repartidor, un trigger ejecuta `pg_notify`. El backend escucha con `LISTEN` y reenvía el aviso por WebSocket al panel de coordinadores, al restaurante y al cliente. Así la actualización en vivo nace en la base de datos.

Capa	Herramienta	Estado
Motor de base de datos	PostgreSQL 16, instalado localmente	Decidido
Administración de la BD	pgAdmin 4	Decidido

Capa	Herramienta	Estado
Diagrama entidad-relación	ERD Tool de pgAdmin, draw.io o dbdiagram.io (solo para dibujar)	Libre
Extensiones de PostgreSQL	pgcrypto para cifrar contraseñas. Sin PostGIS: distancias con función Haversine propia	Decidido
Backend	Python + FastAPI + SQLAlchemy	Decidido
Tiempo real	<code>LISTEN/NOTIFY</code> de PostgreSQL + WebSockets de FastAPI	Decidido
Frontend	Svelte (SvelteKit) + Tailwind CSS, web responsiva para escritorio y celular	Decidido
Mapas	Leaflet + OpenStreetMap (gratis, sin clave de API)	Propuesta
Pagos	Simulados, sin pasarela real	Decidido
Control de versiones	GitHub, un repositorio sin ramas	Decidido
Pruebas de la API	Documentación automática de FastAPI (<code>/docs</code>), Postman o Thunder Client	Libre
Editor de código	VS Code u otro	Libre

Regla para SQLAlchemy: se usa para conectarse y para altas y consultas simples. La lógica del negocio (crear pedido, cambiar estado, asignar repartidor, calcular envío) se llama desde Python como funciones de PostgreSQL, por ejemplo con `text("SELECT fn_asignar_repartidor(:id)")`. Así la lógica queda en la base, que es lo que evalúa la materia. SQLAlchemy no aporta por sí solo geolocalización ni tiempo real: eso lo resuelven la función Haversine y los WebSockets.

11. Modelo de datos

El modelo propuesto tiene 27 tablas (21 del núcleo y 6 de facturación: `tasa_bcv`, `correlativo`, `factura`, `detalle_factura`, `liquidacion_repartidor` y `detalle_liquidacion`) dentro del esquema `deliverexpress`, normalizadas hasta 3FN. Es un punto de partida: el grupo lo revisa en la Fase 1 antes de escribir el DDL.

```
erDiagram
    USUARIO ||--o| CLIENTE : "es"
    USUARIO ||--o| RESTAURANTE : "es"
    USUARIO ||--o| REPARTIDOR : "es"
    USUARIO ||--o| COORDINADOR : "es"
```

```

CATEGORIA ||--o{ RESTAURANTE : "clasifica"
RESTAURANTE ||--o{ HORARIO_RESTAURANTE : "atiende en"
RESTAURANTE ||--o{ RESTAURANTE_ZONA : "cubre"
ZONA ||--o{ RESTAURANTE_ZONA : "es cubierta por"
RESTAURANTE ||--o{ PRODUCTO : "ofrece"
CLIENTE ||--o{ DIRECCION_CLIENTE : "tiene"
ZONA ||--o{ DIRECCION_CLIENTE : "ubica"
ZONA ||--o{ REPARTIDOR : "zona de trabajo"
CLIENTE ||--o{ PEDIDO : "hace"
RESTAURANTE ||--o{ PEDIDO : "recibe"
DIRECCION_CLIENTE ||--o{ PEDIDO : "destino"
REPARTIDOR |o--o{ PEDIDO : "entrega"
ESTADO_PEDIDO ||--o{ PEDIDO : "estado actual"
PEDIDO ||--|{ DETALLE_PEDIDO : "contiene"
PRODUCTO ||--o{ DETALLE_PEDIDO : "aparece en"
PEDIDO ||--|{ HISTORIAL_ESTADO_PEDIDO : "registra"
ESTADO_PEDIDO ||--o{ HISTORIAL_ESTADO_PEDIDO : "describe"
PEDIDO ||--o{ OFERTA_ASIGNACION : "se ofrece"
REPARTIDOR ||--o{ OFERTA_ASIGNACION : "recibe"
REPARTIDOR ||--o{ UBICACION_REPARTIDOR : "reporta"
PEDIDO ||--|| PAGO : "se paga con"
PEDIDO ||--o{ CALIFICACION : "genera"
TARIFA_ENVIO {
    int id_tarifa
    numeric km_desde
    numeric km_hasta
    numeric precio
}
PARAMETRO_SISTEMA {
    varchar clave
    varchar valor
}

```

Diccionario de tablas

Tabla	Guarda	Columnas y restricciones principales
usuario	Credenciales y rol de cada persona que entra al sistema	email UNIQUE, password_hash, rol CHECK (cliente, restaurante, repartidor, coordinador, admin), activo
zona	Barrios o zonas de la ciudad	nombre UNIQUE

Tabla	Guarda	Columnas y restricciones principales
categoria	Tipo de comida (italiana, mexicana, rápida...)	nombre UNIQUE
restaurante	Datos del local	FK usuario y categoria; direccion, telefono, latitud, longitud NUMERIC(9,6); tiempo_prep_min CHECK > 0; calificacion_promedio; activo
horario_restaurante	Horario por día	dia_semana CHECK 0-6, hora_apertura, hora_cierre; UNIQUE (id_restaurante, dia_semana)
restaurante_zona	Zonas a las que entrega cada restaurante (N:M)	PK compuesta (id_restaurante, id_zona)
producto	Menú	FK restaurante; precio NUMERIC(10,2) CHECK > 0; disponible
cliente	Datos del cliente	FK usuario; telefono; calificacion_promedio; en_revision
direccion_cliente	Direcciones de entrega del cliente	FK cliente y zona; direccion, referencia, latitud, longitud; principal
repartidor	Datos y estado del repartidor	FK usuario y zona; tipo_vehiculo CHECK (bicicleta, moto, auto); disponibilidad CHECK (libre, ocupado, desconectado); prioridad CHECK (normal, baja); latitud_actual, longitud_actual; calificacion_promedio; en_revision
coordinador	Personal de la oficina	FK usuario; nombre, telefono
estado_pedido	Catálogo de estados	nombre UNIQUE, orden (1 a 5; cancelado aparte)

Tabla	Guarda	Columnas y restricciones principales
pedido	Cabecera del pedido	FK cliente, restaurante, direccion, repartidor (puede ser NULL), estado; distancia_km, subtotal, costo_envio, propina CHECK ≥ 0 , total, comision_plataforma, monto_restaurante, fecha_creacion
detalle_pedido	Productos del pedido	PK (id_pedido, id_producto); cantidad CHECK > 0 ; precio_unitario copiado del producto
historial_estado_pedido	Cada cambio de estado con su fecha y hora	FK pedido y estado; fecha_hora TIMESTAMPTZ; usuario responsable
oferta_asignacion	Cada vez que se ofrece un pedido a un repartidor	FK pedido y repartidor; respuesta CHECK (pendiente, aceptada, rechazada, expirada); fecha_oferta, fecha_respuesta
ubicacion_repartidor	Posiciones del repartidor para el mapa	FK repartidor y pedido (opcional); latitud, longitud, fecha_hora
tarifa_envio	Precio del envío por rango de km	km_desde, km_hasta CHECK km_hasta $>$ km_desde; precio
pago	Pago simulado con tarjeta	FK pedido UNIQUE; monto; ultimos4 CHAR(4); referencia; estado CHECK (aprobado, rechazado, reembolsado)
calificacion	Calificaciones cruzadas	FK pedido; tipo CHECK (cliente_a_repartidor, cliente_a_restaurante, repartidor_a_cliente); puntaje CHECK 1-5; UNIQUE (id_pedido, tipo)
parametro_sistema	Valores configurables (comisión, umbrales)	clave PK, valor, descripcion

Redundancia controlada: `calificacion_promedio`, `comision_plataforma` y `monto_restaurante` podrían calcularse, pero se guardan. Los promedios evitan recalcular en cada consulta y los mantiene un trigger; los montos conservan el valor histórico aunque la comisión cambie después. Esto se debe justificar en la defensa.

El detalle completo de columnas, restricciones y funciones de facturación está en `roadmap_bd.txt`. Puntos fuertes para la defensa: la factura tiene FK reales al cliente o al restaurante (no un id genérico), un trigger impide modificarla o borrarla, y la numeración usa una tabla con bloqueo de fila para no dejar huecos.

12. Lógica en la base de datos

El corazón del proyecto está en PostgreSQL: el backend solo llama funciones y consulta vistas. Los nombres son propuestas.

Funciones (PL/pgSQL)

Función	Qué hace
<code>fn_distancia_km(lat1, lon1, lat2, lon2)</code>	Distancia en línea recta con la fórmula de Haversine
<code>fn_costo_envio(distancia_km)</code>	Busca el precio en <code>tarifa_envio</code> según el rango
<code>fn_restaurante_disponible(id_restaurante, id_direccion)</code>	Verifica que esté activo, abierto a esa hora y que cubra la zona de la dirección
<code>fn_crear_pedido(cliente, restaurante, dirección, productos JSONB, propina, últimos 4 dígitos)</code>	En una sola transacción: valida, calcula distancia, envío, subtotal, comisión y total; inserta pedido, detalle y pago
<code>fn_asignar_repartidor(id_pedido)</code>	Elige el mejor candidato según RN-07 y RN-08 y crea la oferta
<code>fn_responder_oferta(id_oferta, acepta)</code>	Si acepta: asigna y pone al repartidor "ocupado". Si rechaza: lo registra y busca el siguiente candidato
<code>fn_cambiar_estado(id_pedido, nuevo_estado, id_usuario)</code>	Única vía para cambiar el estado de un pedido
<code>fn_tiempo_estimado(id_pedido)</code>	Minutos estimados: preparación + distancia ÷ velocidad del vehículo

Triggers

Trigger	Tabla y momento	Qué hace
trg_validar_transicion	pedido, BEFORE UPDATE	Rechaza saltos o retrocesos de estado (RN-01)
trg_historial_estado	pedido, AFTER INSERT OR UPDATE	Guarda estado, fecha y hora en <code>historial_estado_pedido</code> (RN-02)
trg_notificar_pedido	pedido, AFTER UPDATE	Envía <code>pg_notify</code> para el tiempo real
trg_liberar_repartidor	pedido, AFTER UPDATE	Al entregar o cancelar, el repartidor vuelve a "libre" (RN-09)
trg_mismo_restaurante	detalle_pedido, BEFORE INSERT	Impide productos de otro restaurante (RN-04)
trg_validar_calificacion	calificacion, BEFORE INSERT	Solo permite calificar pedidos entregados (RN-14)
trg_actualizar_promedio	calificacion, AFTER INSERT	Recalcula el promedio y marca "en revisión" si corresponde (RN-15)
trg_actualizar_prioridad	oferta_asignacion, AFTER UPDATE	Recalcula la tasa de rechazo y la prioridad (RN-10)
trg_ubicacion_actual	ubicacion_repartidor, AFTER INSERT	Actualiza la posición actual del repartidor y envía <code>pg_notify</code> para el mapa

Vistas

Vista	Para qué sirve
vw_pedidos_activos	Panel de coordinadores: pedidos no entregados ni cancelados, con restaurante, repartidor, estado y minutos en el estado actual
vw_repartidores_disponibles	Repartidores libres por zona con su posición y calificación
vw_tiempos_por_etapa	Minutos entre cada estado de cada pedido, usando la función de ventana <code>LAG</code>

Vista	Para qué sirve
vw_desempeno_restaurantes	Tiempo promedio de preparación, pedidos y calificación por restaurante
vw_desempeno_repartidores	Entregas, tiempo promedio, tasa de rechazo y ganancias por repartidor
vw_liquidacion	Cuánto corresponde a la plataforma, a cada restaurante y a cada repartidor por período
vw_recomendaciones_cliente	Categorías y productos que más pide cada cliente

Índices

Cada índice se justifica mostrando `EXPLAIN ANALYZE` antes y después de crearlo.

Índice	Consulta que acelera
pedido (id_estado) parcial, solo pedidos activos	Panel de coordinadores
pedido (id_cliente, fecha_creacion)	Historial del cliente
historial_estado_pedido (id_pedido, fecha_hora)	Tiempos por etapa
repartidor (id_zona, disponibilidad)	Búsqueda de candidatos en la asignación
ubicacion_repartidor (id_repartidor, fecha_hora DESC)	Última posición para el mapa
oferta_asignacion (id_repartidor, fecha_oferta DESC)	Tasa de rechazo de las últimas 20 ofertas

Roles y permisos de la base de datos

Rol de PostgreSQL	Permisos
de_admin	Dueño del esquema; crea y modifica objetos
de_app	Usuario del backend: lee y escribe tablas y ejecuta funciones; no puede borrar tablas ni cambiar el esquema
de_reportes	Solo lectura sobre las vistas, para reportes

Los permisos de cada tipo de usuario final (cliente, repartidor, etc.) se controlan con la columna `usuario.rol` en el backend.

Transacciones y concurrencia

- `fn_crear_pedido` corre en una transacción: si falla el pago o un producto no está disponible, no queda nada guardado.
- `fn_asignar_repartidor` usa `SELECT ... FOR UPDATE SKIP LOCKED` para que dos asignaciones simultáneas nunca tomen al mismo repartidor (RNF-12). Se puede demostrar con dos ventanas del Query Tool de pgAdmin.

Respaldo y datos de prueba

- Respaldo con `pg_dump` en formato personalizado y restauración con `pg_restore`; se demuestra en la presentación.
- Datos de prueba con las cifras reales del cliente: 20 restaurantes, 35 repartidores y 3 coordinadores. Además, alrededor de 50 clientes y 500 pedidos históricos generados con `generate_series`, para que los reportes muestren resultados.

13. Módulos del proyecto

El proyecto se divide en cinco módulos: tres de base de datos (el foco de la materia), uno de backend y uno de frontend. La documentación es transversal: cada módulo documenta lo suyo.

Módulo	Contenido	Entregables
M1 — Modelo de datos y estructura	Diagrama ER, normalización a 3FN, creación del esquema y las 21 tablas con PK, FK, CHECK, UNIQUE y NOT NULL; catálogos (zonas de Puerto Ordaz, categorías, estados, tarifas, parámetros); datos de prueba	DER, justificación de normalización, scripts <code>01_esquema_tablas.sql</code> , <code>02_catalogos.sql</code> , <code>09_datos_prueba.sql</code>

Módulo	Contenido	Entregables
M2 — Pedidos, asignación y logística (BD)	Funciones de distancia, tarifa, disponibilidad, creación de pedido, cambio de estado, asignación y respuesta a ofertas, tiempo estimado; triggers de transición, historial, liberación de repartidor, ubicación y <code>pg_notify</code> ; control de concurrencia	Scripts <code>03_funciones_logistica.sql</code> , <code>04_funciones_pedidos.sql</code> , <code>05_triggers_pedidos.sql</code> , pruebas de cada función
M3 — Pagos, calificaciones, reportes y seguridad (BD)	Registro de pagos y liquidación; calificaciones y triggers de promedios y revisión; tasa de rechazo; las 7 vistas; índices con <code>EXPLAIN ANALYZE</code> ; roles y permisos; respaldo y restauración	Scripts <code>06_pagos_calificaciones.sql</code> , <code>07_vistas.sql</code> , <code>08_indices_roles.sql</code> , comparativa de rendimiento, procedimiento de respaldo
M4 — Backend (FastAPI)	Conexión con SQLAlchemy usando el rol <code>de_app</code> ; inicio de sesión por rol; endpoints para cliente, restaurante, repartidor, coordinador y admin que llaman a las funciones SQL; <code>LISTEN</code> + WebSockets; script simulador de movimiento de repartidores	Código del backend, documentación /docs de FastAPI, lista de endpoints

Módulo	Contenido	Entregables
M5 — Frontend (Svelte + Tailwind)	App del cliente (restaurantes, menú, carrito, pago simulado, propina, seguimiento en mapa, calificar); panel del restaurante; panel del repartidor; panel de coordinadores con mapa y pedidos en vivo; administración básica	Código del frontend, capturas de pantalla para el documento

Orden de dependencia: M1 va primero; M2 y M3 empiezan cuando las tablas están listas; M4 puede avanzar con funciones de prueba mientras M2 y M3 terminan; M5 puede diseñar pantallas con datos falsos desde el inicio.

14. Distribución del equipo

Dos integrantes trabajan en la base de datos, uno en el backend y dos en el frontend. Como la base tiene menos manos, M3 se reparte entre los dos de BD y el del backend ayuda probando funciones y generando datos de prueba.

Integrante	Nombre	Área	Le toca	Documenta
1	Por asignar	Base de datos	M1 completo + facturación en BD + de M3: vistas, índices, roles y respaldo. Consolida el documento final	Secciones 2 a 9, 11 y la parte de vistas, índices y roles de la 12
2	Por asignar	Base de datos	M2 completo + de M3: pagos, liquidación, calificaciones y tasa de rechazo	Sección 12: funciones y triggers
3	Por asignar	Backend	M4 completo; ayuda a M2 probando las funciones desde la API y a M1 con los datos de prueba	Sección 10 y lista de endpoints
4	Por asignar	Frontend	M5: app del cliente y panel del restaurante	Capturas y manual de uso de esas pantallas

Integrante	Nombre	Área	Le toca	Documenta
5	Por asignar	Frontend	M5: panel del repartidor, panel de coordinadores con mapa y administración	Capturas y manual de uso de esas pantallas

Reglas del equipo:

- Los cinco revisan juntos el diagrama ER en la Fase 1; nadie empieza el DDL sin ese acuerdo.
- Todos deben entender el modelo y las funciones principales, porque en la defensa el profesor puede preguntarle a cualquiera.
- Para asignar nombres, conviene poner en M1 a quien mejor domine SQL y diseño, y en M4 y M5 a quienes ya hayan programado web.

15. Plan de trabajo por fases

Sin fecha de entrega definida, el plan se reparte en porcentajes del tiempo total; cuando se sepa la fecha se convierten en semanas.

Fase	Tiempo sugerido	Qué se hace	Quién	Condición para pasar a la siguiente
1. Análisis y diseño	15%	Cerrar los pendientes de la sección 18, dibujar y aprobar el DER, crear el repositorio	Todos	DER aprobado por los 5
2. Estructura de la BD	15%	DDL de tablas, restricciones, catálogos y primeros datos de prueba. En paralelo: esqueleto de FastAPI y maquetas en Svelte	Integrante 1 (2 diseña funciones sobre el DER; 3, 4 y 5 en paralelo)	Cualquier integrante reconstruye la BD desde cero con los scripts

Fase	Tiempo sugerido	Qué se hace	Quién	Condición para pasar a la siguiente
3. Lógica en la BD y API	30%	Funciones, triggers, vistas, índices y roles. El backend expone endpoints que llaman a esas funciones	Integrantes 1, 2 y 3	Un pedido completo recorre los 5 estados llamando a la API
4. Frontend e integración	25%	Pantallas de los cuatro actores, mapa y tiempo real conectados a la API	Integrantes 4 y 5, con 3	Demo de punta a punta: pedir, asignar, entregar, calificar y verlo en el panel
5. Pruebas, documento y defensa	15%	Pruebas con datos masivos, EXPLAIN ANALYZE , respaldo, cierre del documento, ensayo de la presentación	Todos	Documento completo y demo ensayada en la PC de la presentación

16. Convenciones y flujo de trabajo

Como cada integrante tiene su propia base en su PC, la fuente de verdad son los scripts SQL del repositorio, nunca la base de alguien. Nadie cambia tablas desde la interfaz de pgAdmin sin pasarlo a un script.

Estructura del repositorio

```

deliverexpress/
  database/
    00_reset.sql          -- DROP SCHEMA deliverexpress CASCADE;
  CREATE SCHEMA ...
    01_esquema_tablas.sql  -- M1
    02_catalogos.sql      -- M1
    03_funciones_logistica.sql -- M2
    04_funciones_pedidos.sql -- M2
    05_triggers_pedidos.sql -- M2
    06_pagos_calificaciones.sql -- M3
    07_vistas.sql         -- M3
    08_indices_rols.sql   -- M3
    09_datos_prueba.sql   -- M1
    ejecutar_todo.sql     -- \i de 00 a 09 en orden
  backend/               -- M4
  frontend/              -- M5
  docs/                  -- DER exportado, capturas
  README.md              -- cómo instalar y levantar todo

```

Convenciones de nombres en la BD

Elemento	Convención	Ejemplo
Esquema	Uno solo para todo el proyecto	<code>deliverexpress</code>
Tablas	snake_case, singular	<code>detalle_pedido</code>
Clave primaria	<code>id_</code> + nombre de la tabla	<code>id_pedido</code>
Clave foránea	Mismo nombre que la PK que referencia	<code>pedido.id_cliente</code>
Funciones	Prefijo <code>fn_</code>	<code>fn_asignar_repartidor</code>
Triggers	Prefijo <code>trg_</code>	<code>trg_historial_estado</code>
Vistas	Prefijo <code>vw_</code>	<code>vw_pedidos_activos</code>
Índices	<code>idx_</code> + tabla + columnas	<code>idx_pedido_id_estado</code>
Restricciones	<code>pk_</code> , <code>fk_</code> , <code>ck_</code> , <code>uq_</code> + tabla + detalle	<code>ck_calificacion_puntaje</code>
Dinero	<code>NUMERIC(10,2)</code>	<code>costo_envio</code>
Fechas	<code>TIMESTAMPZ</code>	<code>fecha_creacion</code>
Coordenadas	<code>NUMERIC(9,6)</code>	<code>latitud</code>

Flujo de trabajo

1. Antes de trabajar: `git pull` y ejecutar `ejecutar_todo.sql` en pgAdmin para tener la base igual que los demás.
2. Se trabaja todo en `main`, sin ramas. Cada integrante solo edita los archivos de su parte, así no se pisan cambios.
3. Un cambio a una tabla de M1 se avisa al grupo antes de subirlo, porque afecta a todos.
4. Antes de subir un script: la base debe reconstruirse desde cero sin errores.
5. Se sube seguido (al menos una vez por semana), no todo al final.
6. Las credenciales de la base van en un archivo `.env` que no se sube a GitHub.

17. Entregables

Se proponen seis entregables; se ajustan cuando el profesor confirme qué pide.

Entregable	Contenido	Responsable
Documento del proyecto	Este documento completo, con DER final, capturas y resultados de pruebas	Integrante 1 consolida; cada uno aporta su sección
Scripts SQL	Carpeta <code>database/</code> que reconstruye la base completa en PostgreSQL local	M1, M2, M3
Respaldo de la base	Archivo <code>.backup</code> generado con <code>pg_dump</code> , con datos de prueba	M3
Código de la aplicación	Backend y frontend en el repositorio, con README de instalación	M4, M5
Demostración	Flujo completo en vivo: pedido, asignación, estados, mapa, calificación, reportes y consultas en pgAdmin	Todos
Presentación	Explicación del modelo, las reglas en la BD y las decisiones de diseño	Todos

18. Pendientes por definir en el grupo

- ☒ ~~Lenguaje del backend: Python + FastAPI + SQLAlchemy.~~
- ☒ ~~Frontend: Svelte + Tailwind.~~
- ☒ ~~Versión de PostgreSQL: 16.~~
- ☒ ~~Forma de compartir: GitHub sin ramas.~~
- ☒ ~~Supuestos de negocio (sección 9).~~
- ☒ ~~Ciudad de los datos de prueba: Puerto Ordaz.~~
- ☒ ~~Reparto: 2 BD, 1 backend, 2 frontend.~~
- ☐ Fecha de entrega y fechas de entregas parciales, si las hay.
- ☐ Rúbrica del profesor: qué elementos de BD exige y cuánto pesan backend y frontend.
- ☐ Formato del documento final: aún no lo han pedido; se define si el profesor lo indica.
- ☐ Asignar nombres a los integrantes 1 a 5.
- ☐ PC en la que se hará la demostración final.
- ☐ Crear el repositorio en GitHub y dar acceso a los cinco.